



Образовательная программа
"Фундаментальная
математика"

8 июня 2025 г.

Реализация и сравнительный анализ оптимизированных алгоритмов Краскала и Прима

Курсовая работа

Работу выполнил: Турсунов Данил Вячеславович

Научный руководитель: Малышев Дмитрий Сергеевич



Цели и задачи работы

В данной курсовой работе будет рассматриваться задача построения минимального остовного дерева по заданному связному неориентированному взвешенному графу.



Определение

Для связного неориентированного графа $G = (V, E)$ остовное дерево $T = (V, E')$ — это граф, удовлетворяющий условиям:

- 1) T является *деревом*,
- 2) $V(T) = V(G)$ (содержит все вершины исходного графа),
- 3) $E' \subseteq E$ (рёбра принадлежат исходному графу).



Определение

Для взвешенного связного неориентированного графа G минимальное остовное дерево — это остовное дерево $T \subseteq G$, такое что сумма весов рёбер T минимальна среди всех возможных остовных деревьев.



Курсовая работа представляет собой написание программы (кода), которая состоит из 2-х частей: алгоритмической и аналитической. Изначально запускается алгоритмическая часть кода, которая написана на языке программирования C++, в ней реализованы следующие подпрограммы:

- алгоритм Краскалла;
- оптимизация алгоритма Краскалла;
- алгоритм Прима;
- оптимизация алгоритма Прима;
- генератор графов;
- итоговая подпрограмма, которая генерирует графы различного количества вершин и рёбер и запускает на них алгоритмы Краскалла и Прима, замеряя время построения минимального остовного поддерева и выводя в отдельные файлы (для каждой пропорции количества рёбер и количества вершин создаётся отдельный файл) количество вершин и время построения.



Далее запускается аналитическая часть, состоящая из двух подпрограмм, каждая из которых берёт информацию из файлов, сгенерированных алгоритмической частью:

- программа, проводящая сравнительный анализ скорости работы алгоритмов Краскала и Прима. Она строит отдельный график времени, потребовавшегося на построение минимального остовного дерева, в зависимости от количества вершин для каждой пропорции количества рёбер и количества вершин;
- программа, строящая графики отношения времени выполнения алгоритмов Краскала и Прима к оценке их алгоритмической сложности.

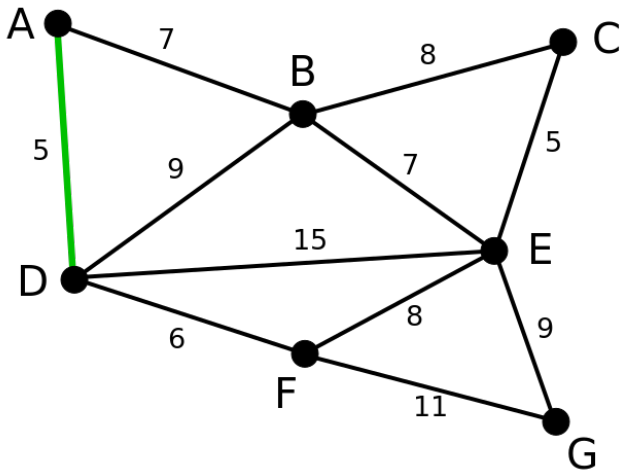


В алгоритме Краскала минимальное остовное дерево строится поэтапно, путём добавления ребёр исходного графа к графу без рёбер, от минимального к максимальному, в случае, если это ребро соединяет вершины, принадлежащие разным компонентам связности строящегося графа.



Пример работы алгоритма Краскала

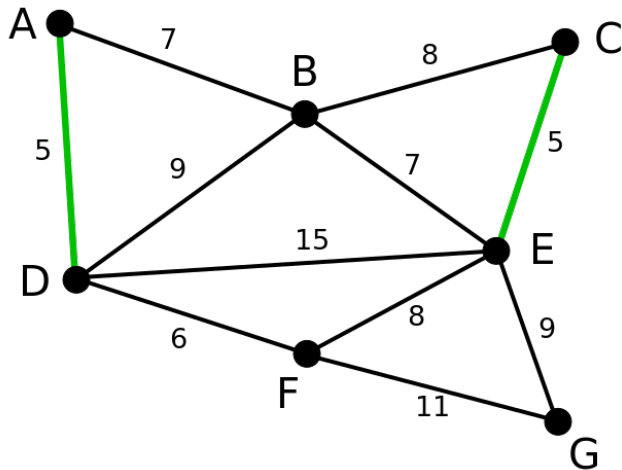
Часть 1





Пример работы алгоритма Краскала

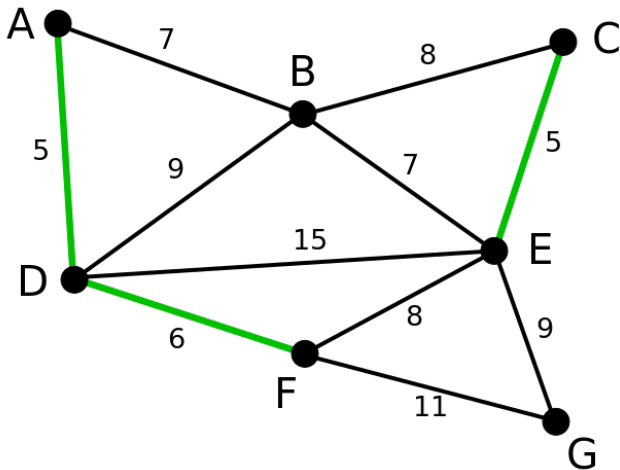
Часть 2





Пример работы алгоритма Краскала

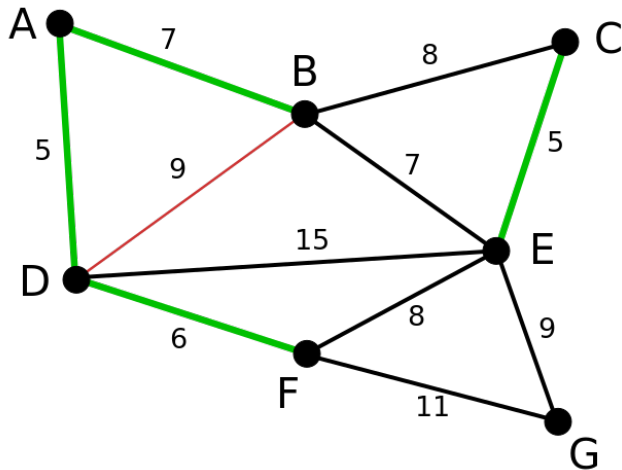
Часть 3





Пример работы алгоритма Краскала

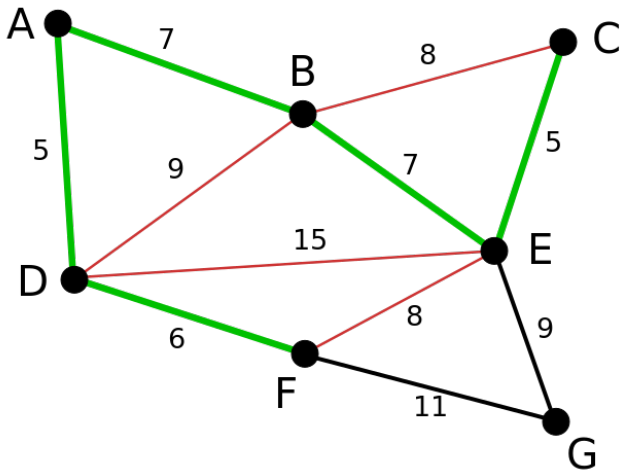
Часть 4





Пример работы алгоритма Краскала

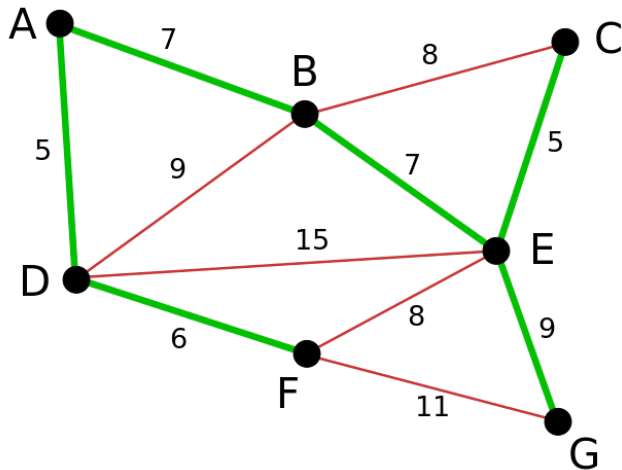
Часть 5





Пример работы алгоритма Краскала

Часть 6





Оптимизация алгоритма Краскалла

Часть 1

В описании алгоритма Краскалла был упущен момент, связанный в проверкой двух вершин на то, соединены ли вершины путём или нет. Для проверки этого факта используется структура данных, называемая системой непересекающихся множеств.



Система непересекающихся множеств представляет собой множество вершин, указывающих на своего предка. То есть, по сути, конкретное множество представляет собой иерархическую структуру, дерево, корнем которого является вершина-представитель множества. Это сделано для того, чтобы множества можно было объединить, указав у вершины-представителя предка (другими словами, подвесив вершину-представителя к другой), который является членом другого множества (необязательно представителем)



Таким образом, для объединения множеств двух вершин необходимо:

- 1) найти вершины-представителей множеств этих двух вершин;
- 2) подвесить вершину-представителя одного множества к вершине-представителю другого множества.



В алгоритмической части программы CHM реализована как класс DSU, у которого есть 2 метода: метод `find_parent` и метод `unite`, которые ищут вершину-представителя конкретной вершины и объединяют множества двух вершин (необязательно представителей) соответственно.



В методе `find_parent` на вход принимается вершина, далее идёт подъём по её предкам вплоть до того момента, когда у вершины не будет предка, а, значит, она является корнем дерева и вершиной-представителем. Соответственно, метод возвращает вершину-представителя.



Метод `unite` принимает на вход 2 вершины, находит у них вершины-представителей и подвешивает одну вершину представителя к другому.



Однако, с такой реализацией скорость объединения множеств может становиться линейной, превращая итоговую скорость работы алгоритма в квадратичную. Для более быстрого объединения будем пользоваться эвристическими подходами.



Первая эвристика заключается в подвешивании менее глубокого дерева к более глубокому. При таком подходе глубина дерева не увеличится (значит, и не увеличится скорость поиска вершины-представителя), кроме случая, когда глубина обоих деревьев одинакова. Для внедрения этой эвристики в программу придётся хранить глубину дерева.



Вторая эвристика заключается в том, чтобы обновлять предка у вершин, через которые мы прошли при поиске вершины представителя. Будем запоминать, через какие вершины мы прошли и, когда дойдём до вершины представителя, подвесим все вершины к вершине-представителю, сокращая таким образом маршрут между вершиной и вершиной-представителем в будущем. При таком подходе глубина перестанет подсчитываться точно и станет оценкой глубины сверху.



После внедрения эвристик в программе, множества будут объединяться практически мгновенно, оценка сложности будет равняться $O(\alpha(n))$, где $\alpha(n)$ - обратная функция Аккермана (настолько медленно растёт, что можно принять за константу).

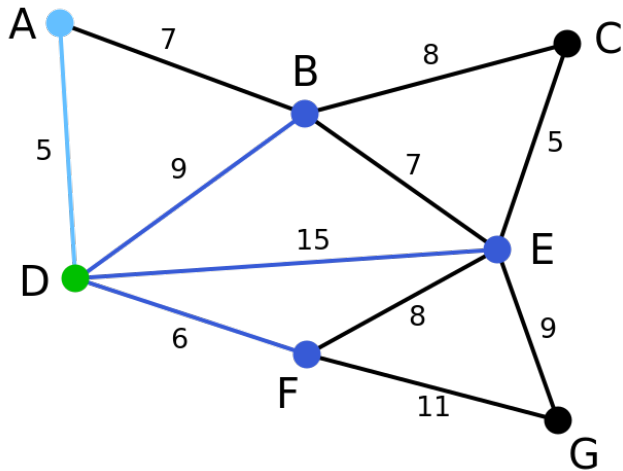


В алгоритме Прима минимальное остовное дерево строится жадно, путём поэтапного добавления рёбер к строящемуся дереву. Очередное добавленное ребро выбирается минимальным из тех, кто соединяет дерево с вершинами, в дерево ещё не добавленные.



Пример работы алгоритма Прима

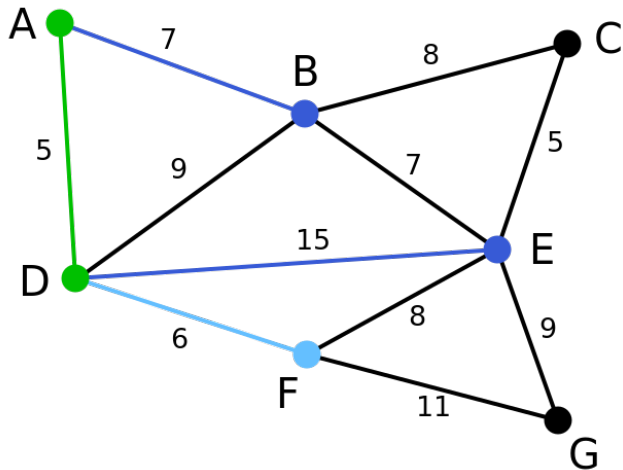
Часть 1





Пример работы алгоритма Прима

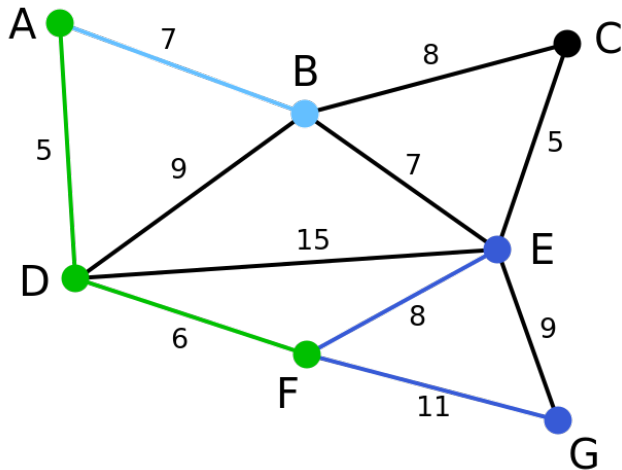
Часть 2





Пример работы алгоритма Прима

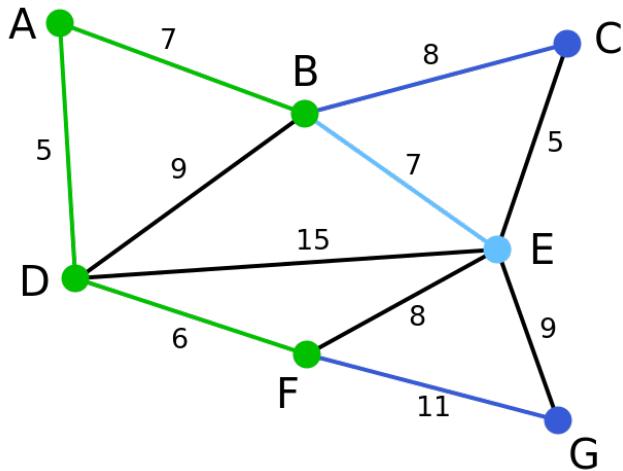
Часть 3





Пример работы алгоритма Прима

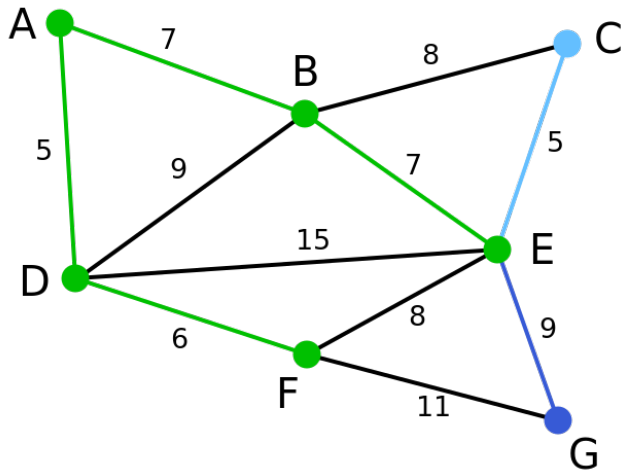
Часть 4





Пример работы алгоритма Прима

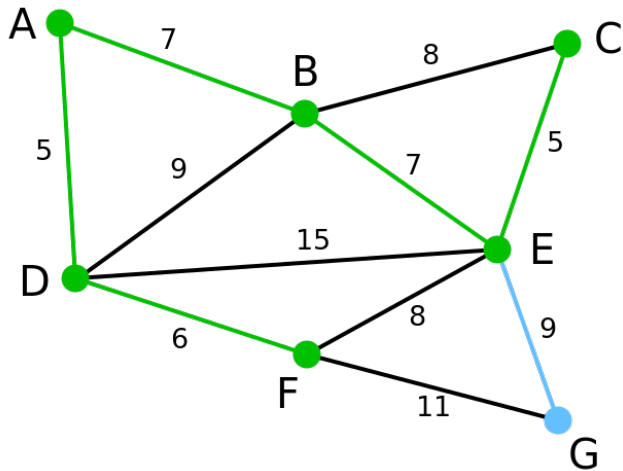
Часть 5





Пример работы алгоритма Прима

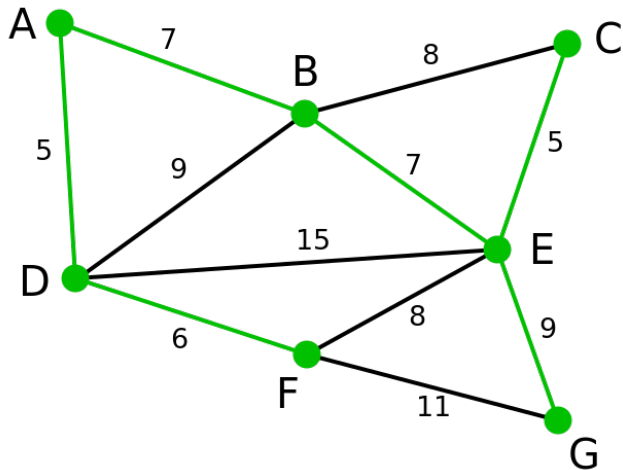
Часть 6





Пример работы алгоритма Прима

Часть 7





В главе выше умышленно был упущен момент выбора очередной вершины, которую планируется добавить в строящееся дерево. Тривиальный алгоритм этой процендуры состоит в том, чтобы циклом пройти по всем ещё не добавленным вершинам и посмотреть все их рёбра, найти среди них то ребро, что соединяет текущую вершину с деревом, и является минимальным среди таких рёбер, то есть просто запомнить минимальное ребро.



Очевидно, что этот алгоритм в худшем случае пройдётся по всем вершинам и по всем рёбрам, что превращает алгоритм Прима в квадратичный относительно количества вершин и рёбер, поэтому в реализованном алгоритме используется оптимизация, основанная на очереди с приоритетами.



Очередь с приоритетами - структура данных, которая является бинарным деревом с одним свойством - значения в вершинах меньше, чем значения в потомках (или больше, чем значения предков). С таким свойством очередь с приоритетами отождествляется с кучей минимумов, в дальнейшем будем использовать именно такой термин.



Куча минимумов позволяет находить минимальное значение практически моментально, в нашем случае, будем находить минимальное ребро, удовлетворяющее условиям. В программе куча минимумов реализована как класс, обладающий полями и методами. Реализована очередь с приоритетами в программе не как дерево, а как список, где поиск левого или правого ребёнка, а также предка, осуществляется по индексам $2i + 1$, $2i + 2$, $(i - 1)/2$, где i - индекс текущей вершины соответственно.



Для кучи минимумов реализованы следующие методы: добавление элемента (он не более чем за $O(\log n)$ займёт нужное место в структуре данных), удаление элемента (тоже занимает $O(\log n)$), а также метод, позволяющий узнать минимальный элемент в куче за $O(1)$. Помимо этого реализованы методы, благодаря которым работают перечисленные выше методы, являющиеся публичными в смысле объектно-ориентированного программирования.



Вернёмся к алгоритму Прима. В цикле, где происходит поиск очередной вершины, вызывается функция `find_min`, которая возвращает подходящую вершину. Эта функция принимает на вход кучу минимумов, состоящую из соседних строящемуся дереву рёбер, и берёт рёбра из этой кучи одно за другим до тех пор, пока не найдёт ребро, соединяющее ещё не использованную вершину с деревом, это ребро и будет являться минимальным среди таких. При таком подходе итоговая сложность работы алгоритма Прима будет $O((n + m)\log n)$.



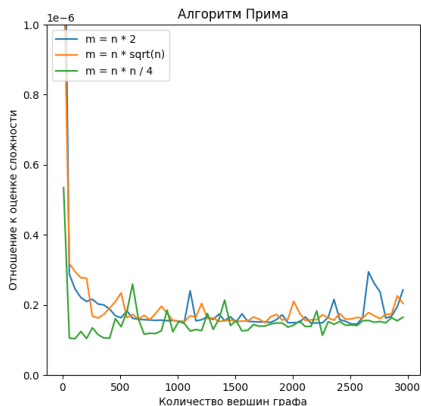
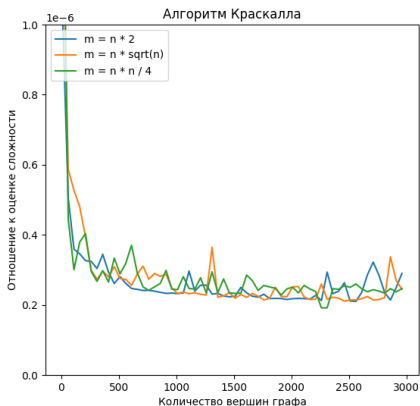
Генерация графов происходит следующим образом: все вершины соединяются рёбрами, образуя круг. Далее случайно выбирается 2 вершины, и, если нет рёбер, соединяющих эти 2 вершины, то ребро добавляется. Если же ребро уже есть, то ещё раз выбираем случайно 2 вершины и проделываем ту же процедуру, и так до тех пор, пока ребро не будет добавлено. Этот алгоритм повторяется, пока не будет добавлено необходимое количество рёбер.



Сравнительный анализ алгоритмов Прима и Краскалла

Часть 1

Отношение времени выполнения к оценке сложности

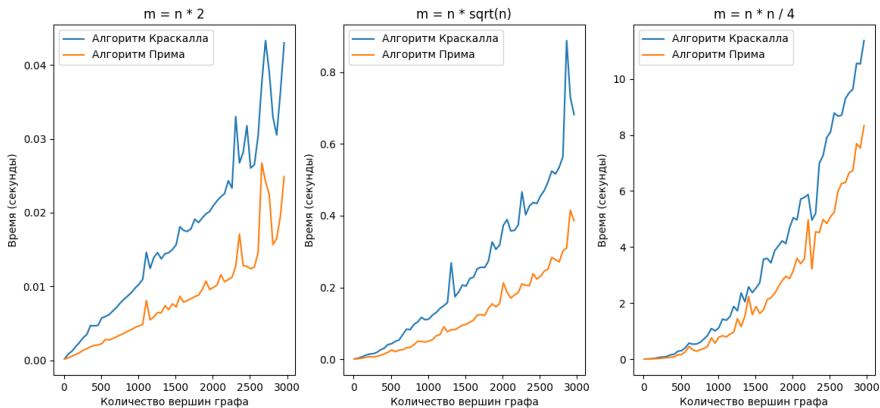




Сравнительный анализ алгоритмов Прима и Краскала

Часть 2

Сравнение алгоритмов Прима и Краскала





Ссылка на репозиторий в GitHub

Реализация алгоритмов Краскала и Прима на языке программирования С++ и код, необходимый для их сравнительного анализа, на языке программирования Python находятся в открытом репозитории на GitHub по следующей ссылке: github.com/dan1lka257/coursework_3



Список литературы

-  Borůvka O. *O jistém problému minimálním*. Práce Moravské Přírodovědecké Společnosti, 1926. Vol. 3, No. 3, pp. 37–58.
-  Иванов А. В. *К истории двух знаменитых оптимизационных алгоритмов в теории графов*. История науки и техники, 2015.
-  Петров С. К. *Алгоритмы Прима и Крускала: сравнительный анализ*. Журнал вычислительной математики, 2018.
-  Охотин А. *Лекция 2: Алгоритмы на графах*. 2024.
-  Сидоров М. Н. *Применение алгоритмов Прима и Крускала в сетевых структурах*. Информационные технологии, 2020.
-  Cormen T. H., Leiserson C. E., Rivest R. L., Stein C. *Introduction to Algorithms*. 3rd ed. MIT Press, 2009.